

Algoritmi de sortare

Bubble · Selection · Insertion · STL · Counting Sort

Andrei Mihai Ciobanu

Algolymp Summer School

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

Rolul sortării

Sortarea este una dintre cele mai frecvente operații în programare.

- Căutarea rapidă a unui element necesită un șir sortat (tehnică studiată în lecția următoare)
- Multe probleme devin simple după sortare: al k-lea cel mai mare element, perechi cu suma dată, eliminarea duplicatelor
- Date reale: clasamente, liste de note, ierarhii

Problema sortării

Dat un vector $v[0..n-1]$, să se rearanjeze elementele astfel încât $v[0] \leq v[1] \leq \ldots \leq v[n-1]$.

Întrebare

Cum se verifică dacă un șir de n numere conține două elemente egale, dacă șirul este nesortat, respectiv sortat?

Nesortat: se compară toate perechile, deci circa $\frac{n(n-1)}{2}$ comparații. Sortat: elementele egale sunt adiacente, deci este suficientă o singură parcurgere, cu $n-1$ comparații.

Inversiuni

O **inversiune** este o pereche de poziții $i < j$ cu $v[i] > v[j]$.

Întrebare

Câte inversiuni are șirul $[5,\ 3,\ 8,\ 1,\ 4]$?

Hint: Se numără, pentru fiecare element, valorile mai mici aflate în dreapta lui.

- 5: $(5,3),\ (5,1),\ (5,4)$ $\rightarrow$ 3 inversiuni
- 3: $(3,1)$ $\rightarrow$ 1 inversiune
- 8: $(8,1),\ (8,4)$ $\rightarrow$ 2 inversiuni
- 1 și 4: nicio inversiune $\Rightarrow$ **6 inversiuni** în total

Legătura cu sortarea: Un șir este sortat dacă și numai dacă nu are inversiuni. Interschimbarea a două elemente **adiacente** aflate în ordine greșită elimină exact o inversiune, iar numărul maxim de inversiuni este $\frac{n(n-1)}{2}$ (șir descrescător).

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

Bubble Sort — idee

Se compară perechile de elemente adiacente, iar elementele aflate în ordine greșită se interschimbă.

Întrebare

Ce se întâmplă cu cel mai mare element după o parcurgere completă, de la stânga la dreapta, a șirului [5, 3, 8, 1, 4]?

5	3	8	1	4	șirul inițial
3	5	8	1	4	$5 > 3$: se interschimbă
3	5	8	1	4	$5 < 8$: rămân pe loc
3	5	1	8	4	$8 > 1$: se interschimbă
3	5	1	4	8	$8 > 4$: se interschimbă
3	5	1	4	8	sfârșitul pasului 1

Concluzie: După pasul 1, cel mai mare element se află pe ultima poziție, la locul său definitiv.

▷ *Vizualizare pas cu pas: visualgo.net/en/sorting*

Bubble Sort — pașii următori

După pasul i, ultimele i elemente sunt la locul lor definitiv, deci pasul $i+1$ nu mai ajunge la ele.

3	5	1	4	8	după pasul 1 (3 interschimbări)
3	1	4	5	8	după pasul 2 (2 interschimbări)
1	3	4	5	8	după pasul 3 (1 interschimbare)

Întrebare

Câte interschimbări a făcut algoritmul în total și ce legătură are numărul cu inversiunile șirului inițial?

$3+2+1=6$ interschimbări, adică exact cele 6 inversiuni ale șirului $[5,3,8,1,4]$.

- Pasul 4 nu mai modifică nimic: pentru n elemente sunt necesari cel mult $n-1$ pași

Bubble Sort — implementare

```
void bubbleSort(int v[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (v[j] > v[j + 1]) {
                swap(v[j], v[j + 1]);
            }
        }
    }
}
```

- Bucla exterioară: $n - 1$ pași
- Bucla interioară: la pasul i se fac $n - 1 - i$ comparații
- Total comparații: $(n-1) + (n-2) + \ldots + 1 = \frac{n(n-1)}{2}$

Întrebare

De ce bucla interioară se oprește la $n - 1 - i$, nu la $n - 1$?

Ultimele i elemente sunt deja la locul lor, deci nu mai este nevoie să fie comparate.

▷ *swap — funcție din STL, interschimbă valorile a două variabile*

Bubble Sort — optimizare și complexitate

Întrebare

Ce complexitate are algoritmul de mai sus pe un șir deja sortat?

$O(n^2)$: comparațiile se fac oricum. Algoritmul se poate opri imediat ce o parcurgere completă nu mai găsește nicio inversare.

```
void bubbleSort(int v[], int n) {
    bool ok = true;          // exista inversiuni
    while (ok) {
        ok = false;          // presupunem ca nu mai sunt
        for (int j = 0; j < n - 1; j++) {
            if (v[j] > v[j + 1]) {
                swap(v[j], v[j + 1]);
                ok = true;   // am gasit o inversiune
            }
        }
    }
}
```

- **Best case** (șir sortat): $O(n)$ — o singură parcurgere, nicio inversare
- **Worst case** (șir sortat invers) și **average case**: $O(n^2)$

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

Selection Sort — idee

La fiecare pas se găsește minimul din partea nesortată și se aduce pe prima poziție a acesteia.

Întrebare

Cum se sortează șirul [5, 3, 8, 1, 4] prin alegerea repetată a minimului?

5	3	8	1	4	minim 1 → se interschimbă cu poziția 0
1	3	8	5	4	minim 3, deja la locul lui
1	3	8	5	4	minim 4 → se interschimbă cu poziția 2
1	3	4	5	8	minim 5, deja la locul lui
1	3	4	5	8	șir sortat

Concluzie: După pasul i, primele $i + 1$ elemente sunt la locul lor definitiv.

▷ *Vizualizare pas cu pas: visualgo.net/en/sorting*

Selection Sort — implementare

```
void selectionSort(int v[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int pozMin = i;                 // pozitia minimului din v[i..n-1]
        for (int j = i + 1; j < n; j++) {
            if (v[j] < v[pozMin]) {
                pozMin = j;
            }
        }
        swap(v[i], v[pozMin]);
    }
}
```

- **Best, worst și average case**: $O(n^2)$ — mereu se fac $\frac{n(n-1)}{2}$ comparații
- Se fac cel mult $n-1$ interschimbări (câte una pe pas), mai puține decât la Bubble Sort

Întrebare

Ce complexitate are algoritmul pe un șir deja sortat?

Tot $O(n^2)$: minimul se caută oricum în toată partea nesortată.

▷ *swap — funcție din STL, interschimbă valorile a două variabile*

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

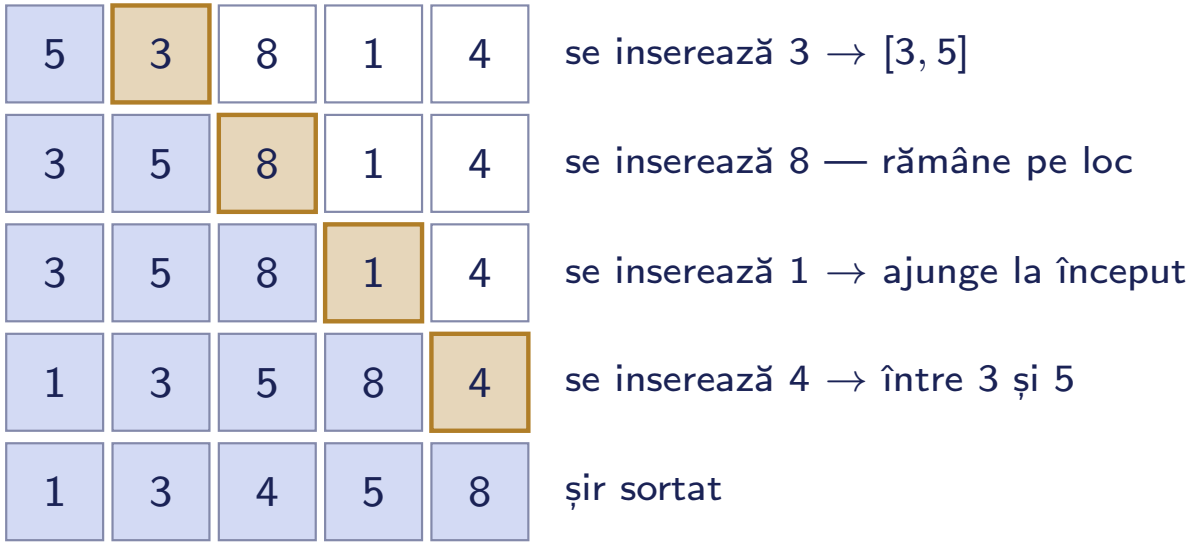
8 Recapitulare

Insertion Sort — idee

Se construiește un prefix sortat, iar fiecare element nou este inserat la locul potrivit în acest prefix.

Întrebare

Cum se extinde un prefix sortat cu încă un element, fără a sorta din nou tot prefixul?



Concluzie: Elementul inserat este mutat spre stânga cât timp este mai mic decât vecinul său.

▷ *Vizualizare pas cu pas: visualgo.net/en/sorting*

Insertion Sort — implementare

```
void insertionSort(int v[], int n) {
    for (int i = 1; i < n; i++) {
        int j = i;
        while (j > 0 && v[j - 1] > v[j]) {
            swap(v[j - 1], v[j]);
            j--;
        }
    }
}
```

- **Best case** (șir sortat): $O(n)$ — nicio deplasare
- **Worst case** (șir sortat invers) și **average case**: $O(n^2)$

Întrebare

De ce condiția buclei conține $j > 0$?

Fără ea, pentru $j = 0$ s-ar accesa $v[-1]$, adică în afara tabloului.

▷ *swap — funcție din STL, interschimbă valorile a două variabile*

Cele trei metode pătratice

Întrebare

Care dintre cele trei metode face cele mai puține interschimbări și care se comportă bine pe un șir aproape sortat?

Algoritm	Best case	Worst case	Interschimbări
Bubble Sort (optimizat)	$O(n)$	$O(n^2)$	egal cu numărul de inversiuni
Selection Sort	$O(n^2)$	$O(n^2)$	cel mult $n-1$
Insertion Sort	$O(n)$	$O(n^2)$	egal cu numărul de inversiuni

- **Selection Sort** face cele mai puține interschimbări
- **Insertion Sort** este eficient pe șiruri aproape sortate: timpul depinde de numărul de inversiuni

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

STL sort — utilizare de bază

C++ oferă funcția `sort` din header-ul `<algorithm>`, cu complexitate $O(n \log n)$.

Întrebare

Cum se sortează un tablou v cu n elemente crescător, respectiv descrescător?

```
#include <algorithm>
using namespace std;

int v[] = {5, 3, 8, 1, 4};
int n = 5;

sort(v, v + n);                  // crescator: 1 3 4 5 8
sort(v, v + n, greater<int>()); // descrescator: 8 5 4 3 1
sort(v + l, v + r + 1);          // doar pozitiile l..r

vector<int> w = {5, 3, 8, 1, 4};
sort(w.begin(), w.end());        // pentru vector
```

▷ *vector, begin, end, greater — din STL*

STL sort — comparatori

Criteriul de sortare poate fi schimbat printr-o **funcție de comparare**, care întoarce `true` dacă primul argument trebuie să apară înaintea celui de-al doilea.

Întrebare

Cum se sortează numerele după valoarea absolută?

```
bool cmpAbs(int a, int b) {
    return abs(a) < abs(b);
}

sort(v, v + n, cmpAbs);
```

Pentru perechi: crescător după primul element, iar la egalitate descrescător după al doilea.

```
bool cmpPair(pair<int, int> a, pair<int, int> b) {
    if (a.first != b.first)
        return a.first < b.first;
    return a.second > b.second;
}

sort(pv, pv + m, cmpPair); // pv: tablou de pair<int, int>
```

▷ *pair, abs — din STL*

STL sort — reguli pentru comparatori

Întrebare

Ce poate face `sort` dacă un comparator întoarce `true` și pentru două elemente egale, de exemplu `return a <= b;`?

Comportament nedefinit: programul poate da rezultate greșite sau se poate opri cu eroare (acces în afara tabloului).

Reguli pentru un comparator corect

- `cmp(a, a)` trebuie să întoarcă `false`: se folosesc `<` și `>`, niciodată `<=` sau `>=`
- dacă `cmp(a, b)` este `true`, atunci `cmp(b, a)` trebuie să fie `false`
- relația trebuie să fie tranzitivă: din `cmp(a, b)` și `cmp(b, c)` rezultă `cmp(a, c)`

- `sort` nu garantează ordinea relativă a elementelor egale; `stable_sort` o păstrează (sortare **stabilă**)

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

Counting Sort — idee

Elementele nu se compară între ele: se numără de câte ori apare fiecare valoare, apoi șirul se reconstruiește.

Întrebare

Cum se poate sorta șirul $v = [3,\ 1,\ 4,\ 1,\ 5,\ 3,\ 3]$ cu valori în intervalul $[0, 5]$, fără nicio comparație între elemente?

Hint: Valorile sunt numere mici: pot fi folosite ca indici într-un tablou de contoare.

Valoare	0	1	2	3	4	5
Contor	0	2	0	3	1	1

Rezultat: Fiecare valoare se scrie de câte ori apare: $[1, 1, 3, 3, 3, 4, 5]$.

▷ *Vizualizare pas cu pas: visualgo.net/en/sorting*

Counting Sort — implementare

```
const int MAXV = 1000000;
int cnt[MAXV + 1];                  // contoarele, initial 0

void countingSort(int v[], int n) {
    int maxVal = 0;
    for (int i = 0; i < n; i++) {
        cnt[v[i]]++;                // numaram aparitiile
        if (v[i] > maxVal) maxVal = v[i];
    }
    int idx = 0;                    // reconstruim sirul sortat
    for (int val = 0; val <= maxVal; val++) {
        for (int i = 0; i < cnt[val]; i++) {
            v[idx++] = val;
        }
        cnt[val] = 0;               // contoarele raman 0 pentru apelul urmator
    }
}
```

- Complexitate $O(n + k)$: n pentru numărare, k pentru parcurgerea valorilor posibile

Counting Sort — limitări

Întrebare

În ce situații Counting Sort nu este potrivit?

Când funcționează

- Valori întregi nenegative
- k mic (de exemplu note 1–10, vârste 0–150, valori $\leq 10^6$)

Când nu funcționează

- Valori negative (se adună o constantă tuturor valorilor) sau reale (trebuie transformate mai întâi)
- k foarte mare: valori până la 10^9 ar necesita un tablou de aproximativ 4 GB

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

Comparația algoritmilor de sortare

Algoritm	Timp	Spațiu suplimentar	Condiții
Bubble Sort	$O(n^2)$	$O(1)$	orice tip comparabil
Selection Sort	$O(n^2)$	$O(1)$	orice tip comparabil
Insertion Sort	$O(n^2)$	$O(1)$	orice tip comparabil
STL sort	$O(n \log n)$	$O(\log n)$	orice tip comparabil
Counting Sort	$O(n + k)$	$O(k)$	întregi în $[0, k]$

Întrebare

Ce algoritm este potrivit pentru $n = 10^5$ valori până la 10^9?

STL sort: metodele pătratice fac circa 10^{10} operații, iar Counting Sort ar necesita un tablou de 10^9 elemente.

- În practică se folosește **STL sort**, dacă nu există restricții speciale
- Counting Sort este mai rapid decât STL sort când $k = O(n)$
- Bubble Sort este util didactic, dar se evită în concursuri

Cuprins

1 Problema sortării

2 Bubble Sort

3 Selection Sort

4 Insertion Sort

5 STL sort

6 Counting Sort

7 Comparație

8 Recapitulare

Recapitulare

Operație	Cod C++
Sortare crescătoare	`sort(v, v + n)`
Sortare descrescătoare	`sort(v, v + n, greater<int>())`
Sortare a pozițiilor $l..r$	`sort(v + l, v + r + 1)`
Comparator propriu	`sort(v, v + n, cmp)`

- **Bubble, Selection, Insertion Sort**: $O(n^2)$; Bubble și Insertion fac exact atâtea interschimbări câte inversiuni există
- **STL sort**: $O(n \log n)$, soluția implicită; comparatorul nu trebuie să întoarcă `true` pentru elemente egale
- **Counting Sort**: $O(n + k)$, doar pentru întregi mici

Probleme de antrenament

Ușoară

1. **Even Sort** (AlgoCode #1081)

Medii

1. **K Sort** (AlgoCode #1077)
2. **Sort Primes** (AlgoCode #1076)

Grea

1. **Array Task** (AlgoCode #1082)

Resurse

Materiale

1. **pbinfo.ro** — Sortarea tablourilor (RO)
2. **pbinfo.ro** — Sortare prin selecție (RO)
3. **pbinfo.ro** — STL în română
4. **cp-algorithms.com** — Bubble Sort (EN)
5. **cppreference.com** — std::sort (EN)
6. **wikipedia.org** — Quicksort și Merge Sort, algoritmi $O(n \log n)$ (EN)

Vizualizări

1. **VisuAlgo** — animații pas cu pas pentru Bubble, Selection, Insertion, Merge, Quick, Counting și Radix Sort, cu date introduse de utilizator (EN)
2. **Toptal** — opt algoritmi comparați pe date aleatoare, aproape sortate, inversate și cu puține valori distincte (EN)
3. **USFCA** — Comparison Sorting Visualization (EN)
4. **Sound of Sorting** — vizualizare și sunet pentru operațiile de sortare (EN)
5. **YouTube** — 15 Sorting Algorithms in 6 Minutes (EN)